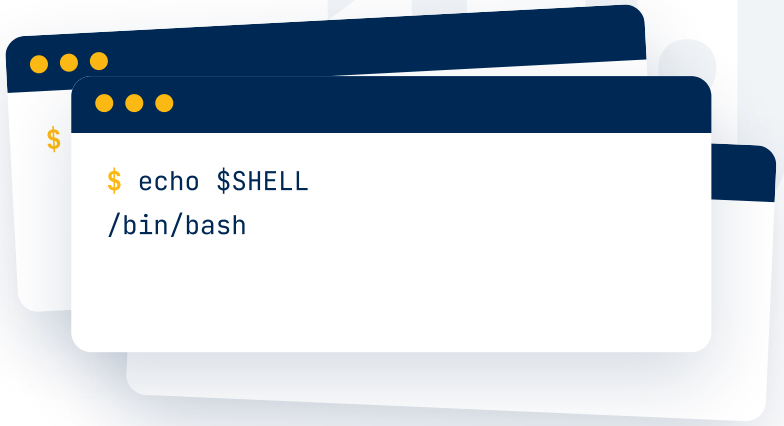
LPIC-1 · 102-500

Tema 105
Shells y Scripts

Personalización del entorno de shell y scripting en Bash



SUBTEMA

105.1 — Customizar y usar el entorno shell

PESO 4

SUBTEMA

105.2 — Escribir scripts sencillos

PESO 4

Material original, no oficial · Referencia: lpi.org

Índice

Estructura del tema y bloques prácticos



1

SUBTEMA

105.1 — Entorno Shell

Login vs no-login, /etc/skel, PATH

2

SUBTEMA

105.2 — Scripts Sencillos

Shebang, test, if/for/while, variables

3

PRÁCTICA

Lab guiado

~/bin/hola105 + check.sh

4

AUTOEVALUACIÓN

Quiz de autoevaluación

5 preguntas tipo test

PESO 4 · 105.1

PESO 4 · 105.2

TOTAL TEMA: PESO 8

105.1 · ENTORNO SHELL · PESO 4

Shell de login vs shell de no-login

CONCEPTO

- **Login shell:** se autentica al iniciar sesión.
- **No-login:** se abre dentro de una sesión activa.
- Login lee `/etc/profile` y `~/.bash_profile`.
- No-login solo lee `~/.bashrc`.
- `/etc/bash.bashrc` aplica a ambos en muchas distros.
- Cierra sesión con `exit` o `logout`.

ORDEN DE LECTURA (LOGIN SHELL)

1. **`/etc/profile`**
   Sistema global

▼

2. **`~/.bash_profile`**
   Usuario (login)

▼

3. **`~/.bashrc`**
   Suele llamarlo .bash_profile

```
$ source ~/.bashrc  # recargar sin reiniciar
```

105.1 · ENTORNO SHELL · PESO 4

Comandos clave del entorno shell

Cómo definir, exportar y reutilizar variables y funciones

Comando	Función	Ejemplo
`export`	Marca variable como de entorno (visible a subshells)	`export PATH=$PATH:~/bin`
`set`	Lista todas las variables y funciones shell	`set | grep USER`
`unset`	Elimina variable o función del shell	`unset MI_VAR`
`alias`	Crea atajo de comando	`alias ll='ls -la'`
`function`	Define función reutilizable	`function saluda(){ echo $1; }`
`source (.)`	Ejecuta script en el shell actual	`source ~/.bashrc`

TIP

`export` cambia el entorno de procesos hijo; sin `export`, la variable queda solo en el shell actual.

CUIDADO

Los alias no persisten tras cerrar sesión si no se guardan en `~/.bashrc`.

105.1 · ENTORNO SHELL · PESO 4

/etc/skel y la variable PATH

· /etc/skel

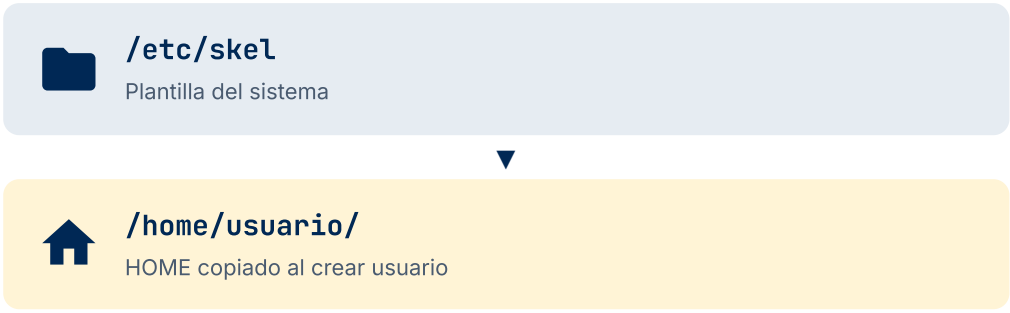
- Plantilla para nuevos usuarios (`useradd`).
- Se copia al crear el HOME del usuario.
- Suele contener `.bashrc` , `.bash_profile` , `.profile` .
- Personaliza la shell por defecto de nuevos usuarios.

```
# /etc/skel/.bashrc ejemplo
export EDITOR=nano
alias ll='ls -la'
```

· PATH

- Variable de entorno con directorios ejecutables.
- Se separa con dos puntos `:` entre rutas.
- Orden de búsqueda: izquierda a derecha.
- Añade ruta con: `export PATH=$PATH:~/bin`

PLANTILLA DE USUARIO



VARIABLE PATH

`/usr/bin` : `/bin` : `/usr/local/bin` : `~/bin`

Búsqueda de izquierda a derecha · `~/bin` añadido por el usuario

105.1 · ENTORNO SHELL · PESO 4

Ejemplo práctico de personalización

Configurar prompt, alias y función para nuevos usuarios

~/.bash_profile

```
# ~/.bash_profile — login shell
export PATH=$PATH:$HOME/bin
export EDITOR=vim
export PS1='\u@\h:\w$ '

# Alias útiles
alias ll='ls -la --color=auto'
alias ..='cd ..'

# Función de bienvenida
function bienvenida() {
  echo "Hola $USER, son las $(date +%H:%M)"
}

# Cargar .bashrc si existe
if [ -f ~/.bashrc ]; then
  source ~/.bashrc
fi
```

OBJETIVO

Definir entorno personalizado para login shell, reutilizable en cualquier sesión nueva.

APLICAR

`cp /etc/skel/.bash_profile ~` y luego `source ~/.bash_profile` para recargar.

VERIFICAR

Ejecuta `env | grep -E 'PATH|EDITOR'` y `type bienvenida` para confirmar.

105.2 · SCRIPTS SENCILLOS · PESO 4

Shebang y permisos de ejecución

Cómo convertir un archivo de texto en un script ejecutable

· SHEBANG (#!)

- Primera línea del script.
- Indica el intérprete a usar.
- `#!/bin/bash` para scripts Bash.
- `#!/usr/bin/env bash` más portable.
- Sin shebang, se ejecuta en el shell actual.

· PERMISOS DE EJECUCIÓN

- Sin `+x`, el script no puede ejecutarse directamente.
- Se otorga con: `chmod +x script.sh`
- Verificar con: `ls -l script.sh`
- Ejecutar: `./script.sh` o `bash script.sh`

```
#!/bin/bash
# script.sh — ejemplo shebang

echo "Hola desde $0"
exit 0
```

· FLUJO DE CREACIÓN

`touch script.sh` → `chmod +x script.sh` → `./script.sh`

105.2 · SCRIPTS SENCILLOS · PESO 4

Evaluación de condiciones: test, [ ] y [[ ]]

Construcción	Tipo	Características	Ejemplo
`test EXPR`	POSIX	Comando externo, portable	`test -f /etc/hosts && echo OK`
`[ EXPR ]`	POSIX	Alias de `test`; requiere espacios internos	`[ -d /tmp ] && echo "existe"`
`[[ EXPR ]]`	Bash/Ksh	Built-in; soporta patrones y &&, || sin escape	`[[ $s == h* ]] && echo coinciden`
`(( EXPR ))`	Bash	Evaluación aritmética entera	`(( x > 5 )) && echo mayor`

OPERADORES COMUNES

- `-f` archivo regular
- `-d` directorio
- `-r` legible
- `-w` escribible
- `-x` ejecutable
- `-z` cadena vacía
- `=` igual cadena
- `!=` distinto
- `-eq` igual entero
- `-ne` distinto entero

CUIDADO CON LOS ESPACIOS

En `[ -f file ]` los espacios son obligatorios tras `[` y antes de `]`. Omitirlos provoca error de sintaxis. `[[ ]]` es más permisivo y permite agrupar con `&&` sin escapar.

105.2 · SCRIPTS SENCILLOS · PESO 4

Estructuras de control: if, for, while

Con read y seq para iteración y entrada de datos

· IF / ELIF / ELSE

```
if [ "$1" = "--help" ]; then
  echo "Uso: $0 [arg]"
elif [ -z "$1" ]; then
  echo "Sin argumentos"
else
  echo "Arg: $1"
fi
```

Condicional clásico con [ ].

· FOR + SEQ

```
for i in $(seq 1 5); do
  echo "Iteración $i"
done

# Otra forma
for f in *.txt; do
  echo "$f"
done
```

seq genera listas numéricas.

· WHILE + READ

```
while read linea; do
  echo "Leído: $linea"
done < archivo.txt

# Bucle con contador
n=0
while [ $n -lt 3 ]; do
  n=$((n+1))
done
```

read consume línea por línea.

`seq INI FIN` `read VAR` `(( expr ))`

105.2 · SCRIPTS SENCILLOS · PESO 4

Operadores de control y variables especiales

· OPERADORES DE CONTROL

- `cmd1 && cmd2` — ejecuta cmd2 solo si cmd1 tuvo éxito.
- `cmd1 || cmd2` — ejecuta cmd2 solo si cmd1 falló.
- `cmd1 ; cmd2` — ejecuta ambos secuencialmente.
- `exec cmd` — reemplaza el shell actual por cmd.
- `exec > archivo` — redirige toda la salida del script.

```
[ -f /etc/passwd ] && echo OK || echo FALTA
exec 2>>errores.log
```

· VARIABLES ESPECIALES

Variable	Significado
$0	Nombre del script
$1 … $9	Argumentos posicionales 1-9
$#	Número total de argumentos
$@	Todos los argumentos (cada uno entre comillas)
$*	Todos los argumentos como una sola cadena
$?	Código de salida del último comando
$$	PID del shell actual
$!	PID del último proceso en background

```
./script.sh a b c
echo "$0 $1 $# $@ $?"
```

REFERENCIA RÁPIDA

Comandos clave del tema 105

Resumen ejecutivo para repaso antes del examen

Comando	Subtema	Uso breve
`export VAR=val`	105.1	Exporta variable al entorno de subshells
`alias ll='ls -la'`	105.1	Define atajo de comando (no persiste solo)
`source script.sh (o .)`	105.1	Ejecuta script en el shell actual
`chmod +x script.sh`	105.2	Otorga permiso de ejecución
`test EXPR / [ ] / [[ ]]`	105.2	Evalúa condiciones y devuelve 0/1
`read VAR`	105.2	Lee una línea de stdin y la asigna a VAR
`seq 1 5`	105.2	Genera secuencia numérica (1 2 3 4 5)
`exec cmd`	105.2	Reemplaza el proceso shell actual por cmd

105.1 — ENTORNO | 105.2 — SCRIPTS | PESO TOTAL TEMA: 8

Material original, no oficial · Referencia genérica a lpi.org · No se reproducen textos ni logos oficiales LPI.

LAB PRÁCTICO

Lab: crea ~/bin/hola105 y check.sh

Cinco pasos para tener tu primer script funcional

1 CREAR DIRECTORIO PERSONAL

`mkdir -p ~/bin`

Añade ~/bin a PATH si no está.

2 AÑADIR AL PATH

`echo 'export PATH=$PATH:~/bin' >> ~/.bashrc`

Recarga con `source ~/.bashrc`.

3 CREAR hola105

`nano ~/bin/hola105` con contenido:

```
#!/bin/bash
echo "Hola $USER, soy $0"
```

4 DAR PERMISO DE EJECUCIÓN

`chmod +x ~/bin/hola105`

Verifica con `ls -l ~/bin/hola105`.

5 CREAR check.sh

`nano ~/bin/check.sh` con shebang, if [-f], for en *.log, salida con $?.

Ver bloque de código adjunto.

```
#!/bin/bash
# check.sh — verifica logs
LOGDIR=${1:-/var/log}

if [ ! -d "$LOGDIR" ]; then
  echo "Error: $LOGDIR no existe"
  exit 1
fi

for f in "$LOGDIR"/*.log; do
  [ -f "$f" ] && echo "OK: $f"
done

echo "Total revisado: $?"
```

EJECUTAR

```
chmod +x ~/bin/check.sh && check.sh
check.sh /tmp
```

AUTOEVALUACIÓN

Quiz — 5 preguntas tipo test

Marca la respuesta correcta y lee la explicación

5 / 5 PREGUNTAS

Q1 ¿Qué hace `export MI_VAR=valor`?

A) Crea una variable local en el shell actual

B) **Define una variable de entorno visible en subshells** ✓ CORRECTA

C) Elimina la variable MI_VAR

D) Imprime el valor de MI_VAR

`export` marca la variable como de entorno, permitiendo que los procesos hijo la hereden.

Q2 Diferencia entre `source script.sh` y `./script.sh`:

A) Ninguna, son equivalentes

B) **`./` ejecuta en subshell; `source` ejecuta en el shell actual** ✓ CORRECTA

C) `./` requiere shebang; `source` no

D) `source` crea un nuevo proceso

`source` (o `.`) ejecuta el script en el shell actual, conservando variables y funciones.

Q3 ¿Qué hace `/etc/skel`?

A) Define el shell por defecto del sistema

B) **Plantilla copiada al HOME de nuevos usuarios** ✓ CORRECTA

C) Almacena ejecutables del sistema

D) Configura variables de red

`useradd` copia el contenido de /etc/skel al HOME del nuevo usuario.

Q4 En `[ -f archivo ]`, ¿qué pasa si omites los espacios?

A) Funciona igual

B) **Error de sintaxis: `[` exige espacio antes y después** ✓ CORRECTA

C) Se evalúa como falsa

D) Se convierte en `[[ ]]`

`[` es un comando, no una sintaxis: requiere espacios internos obligatorios.

Q5 Diferencia entre `"$@"` y `$*`:

A) Son idénticos

B) **`$@` expande cada argumento entre comillas; `$*` los une en una sola cadena** ✓ CORRECTA

C) `$*` solo funciona en Bash

D) `$@` ignora el primer argumento

Con `"$@"` cada argumento queda protegido como palabra separada; `"$*"` los une con IFS.

5

PREGUNTAS

Repasa las falladas antes del examen LPIC-1

¡Tema 105 completado! Sigue practicando en la terminal

Has cubierto los pesos 4 + 4 del tema 105 (Shells y Scripts). La mejor manera de prepararte para el examen LPIC-1 102-500 es ejecutar los comandos en una terminal real y construir tus propios scripts.

Material oficial de aprendizaje

`https://learning.lpi.org/es/learning-materials/102-500/`

Guías oficiales en español (LPI Learning Materials)

Objetivos del examen 101-102

`https://www.lpi.org/es/exam-101-102-objectives/`

Lista oficial de objetivos del examen LPIC-1

```
bash — 105

$ ./hola105
Hola user, soy hola105
$ check.sh
```

PESO 4 + 4

= tema 105 · peso 8

MATERIAL ORIGINAL, NO OFICIAL

Esta presentación es material educativo independiente y no está afiliada ni respaldada por Linux Professional Institute. Las marcas y nombres LPI pertenecen a sus titulares.